Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 5

по курсу «Методы оптимизации»

«Методы первого порядка»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Реализовать методы первого порядка: Дайяна, Полака-Ривьера, Диксона, Хестена-Стифеля.

2 Результаты

Исходный код программы представлен в листинге 1.

```
using Plots
using LinearAlgebra
using Printf
using ForwardDiff

gr()

function f(x::Vector, a::Real, b::Real)::Float64
    return a * x[1]^2 + b * x[2]^2
end

function grad_f(x::Vector, a::Real, b::Real)::Vector
    return [2 * a * x[1], 2 * b * x[2]]
end

function f_rosenbrock(x::Vector)
    return (1 - x[1])^2 + 100 * (x[2] - x[1]^2)^2
end

function grad_f_rosenbrock(x::Vector)::Vector
    g1 = -2 * (1 - x[1]) - 400 * x[1] * (x[2] - x[1]^2)
    g2 = 200 * (x[2] - x[1]^2)
    return [g1, g2]
end

function f_schwefel(x::Vector)
    return 418.9829 * length(x) - sum(xi -> xi * sin(sqrt(abs(xi)))
    , x)
end

function grad_f_schwefel(x::Vector)::Vector
    g = similar(x)
    for i in eachindex(x)
        xi = x[i]
```

```
        s = sqrt(abs(xi)) + 1e-20
        if xi >= 0
            g[i] = -sin(s) - xi * cos(s) / (2 * s)
        else
            g[i] = -sin(s) + xi * cos(s) / (2 * s)
        end
    end
    return g
end

function f_rastrigin(x::Vector)
    return 10 * length(x) + sum(xi -> xi^2 - 10 * cos(2   * xi), x)
end

function grad_f_rastrigin(x::Vector)::Vector
    return [2 * xi + 20   * sin(2   * xi) for xi in x]
end

function exact_line_search(x::Vector, d::Vector, grad::Vector, a::
    Real, b::Real)::Float64
    numerator = -dot(grad, d)
    Hd = [2 * a * d[1], 2 * b * d[2]]
    denominator = dot(d, Hd)
    if abs(denominator) < 1e-14
        return 1e-8
    end
    alpha = numerator / denominator
    return max(alpha, 1e-8)
end

function backtracking_line_search(x::Vector,
                                  d::Vector,
                                  grad::Vector,
                                  func::Function;
                                  alpha0::Float64 = 0.5,
                                  rho::Float64 = 0.5,
                                  c::Float64 = 1e-4,
                                  max_iter::Int = 50)::Float64
    descent = dot(grad, d)
    if descent >= 0
        return 0.0
    end

    alpha = alpha0
    f_current = func(x)
```

```
    for _ in 1:max_iter
        x_new = x + alpha * d
        f_new = func(x_new)

        if f_new <= f_current + c * alpha * descent
            return alpha
        end
        alpha *= rho
        if alpha < 1e-12
            break
        end
    end

    return alpha
end

function conjugate_gradient_quadratic(x0::Vector, a::Real, b::Real;
                                      method::Symbol=:polak_ribiere,
                                      max_iter::Int=1000,
                                      tol::Real=1e-8)
    history = [copy(x0)]
    directions = []
    x = copy(x0)
    grad = grad_f(x, a, b)
    if norm(grad) < tol
        return x, history, directions
    end
    d = -grad
    push!(directions, copy(d))
    grad_prev = copy(grad)
    d_prev = copy(d)
    for iter in 1:max_iter
        alpha = exact_line_search(x, d, grad, a, b)
        x = x + alpha * d
        push!(history, copy(x))
        grad_new = grad_f(x, a, b)
        if norm(grad_new) < tol
            break
        end
        grad_diff = grad_new - grad
        beta = 0.0
        if method == :polak_ribiere
            numerator = dot(grad_new, grad_diff)
            denominator = dot(grad, grad)
            beta = numerator / denominator
```

```
            beta = max(beta, 0.0)
        elseif method == :hestenes_stiefel
            numerator = dot(grad_new, grad_diff)
            denominator = dot(d_prev, grad_diff)
            if abs(denominator) > 1e-14
                beta = numerator / denominator
            end
        elseif method == :dixon
            numerator = dot(grad_new, grad_new)
            denominator = dot(d_prev, grad_diff)
            if abs(denominator) > 1e-14
                beta = numerator / denominator
            end
        elseif method == :dai_yuan
            numerator = dot(grad_new, grad_diff)
            denominator = dot(d_prev, grad_diff)
            if denominator > 1e-14
                beta = numerator / denominator
            else
                beta = 0.0
            end
        end
        d_used = copy(d)
        d = -grad_new + beta * d
        push!(directions, copy(d))
        grad_prev = copy(grad)
        d_prev = d_used
        grad = grad_new
    end
    return x, history, directions
end

function check_conjugacy(directions::Vector, a::Real, b::Real)
    A = [2a 0; 0 2b]
    max_val = 0.0
    for i in 1:length(directions)
        for j in (i+1):length(directions)
            val = dot(directions[i], A * directions[j])
            max_val = max(max_val, abs(val))
            if abs(val) > 1e-6
                return false, max_val
            end
        end
    end
    return true, max_val
```

```
end

function conjugate_gradient_general(x0::Vector,
                                    func::Function,
                                    grad_func::Function;
                                    method::Symbol = :polak_ribiere,
                                    max_iter::Int = 1000,
                                    tol::Real = 1e-8)
    history = [copy(x0)]
    directions = []
    x = copy(x0)
    grad = grad_func(x)
    if norm(grad) < tol
        return x, history, directions
    end
    d = -grad
    push!(directions, copy(d))
    for iter in 1:max_iter
        alpha = backtracking_line_search(x, d, grad, func)
        if alpha == 0.0
            d = -grad
            alpha = backtracking_line_search(x, d, grad, func)
        end
        x_new = x + alpha * d
        push!(history, copy(x_new))
        grad_new = grad_func(x_new)
        if norm(grad_new) < tol
            return x_new, history, directions
        end
        grad_diff = grad_new - grad
        d_old = d
        beta = 0.0

        if method == :polak_ribiere
            beta = dot(grad_new, grad_diff) / dot(grad, grad)
            beta = max(beta, 0.0)

        elseif method == :hestenes_stiefel
            denom = dot(d_old, grad_diff)
            if abs(denom) > 1e-14
                beta = dot(grad_new, grad_diff) / denom
                beta = max(beta, 0.0)
            end

        elseif method == :dixon
```

```
            denom = dot(d_old, grad)
            if abs(denom) > 1e-14
                beta = dot(grad_new, grad_new) / denom
                beta = max(beta, 0.0)
            end

        elseif method == :dai_yuan
            denom = dot(d_old, grad_diff)
            if abs(denom) > 1e-14
                beta = dot(grad_new, grad_new) / denom
            end
        end

        d = -grad_new + beta * d_old
        push!(directions, copy(d))
        x = x_new
        grad = grad_new
    end
    return x, history, directions
end

function check_conjugacy_general(directions::Vector, history::Vector, hess_func::Function; tol=1e-4)
    max_val = 0.0
    for i in 1:length(directions)
        for j in (i+1):length(directions)
            x_ref = length(history) >= i ? history[i] : history[end]
            H = hess_func(x_ref)
            val = dot(directions[i], H * directions[j])
            max_val = max(max_val, abs(val))
            if abs(val) > tol
                return false, max_val
            end
        end
    end
    return true, max_val
end

function steepest_descent_quadratic(x0::Vector, a::Real, b::Real; max_iter::Int=1000, tol::Real=1e-8)
    history = [copy(x0)]
    grads = []
    x = copy(x0)
    grad = grad_f(x, a, b)
```

```
    push!(grads, copy(grad))
    for _ in 1:max_iter
        if norm(grad) < tol
            break
        end
        d = -grad
        alpha = exact_line_search(x, d, grad, a, b)
        x = x + alpha * d
        push!(history, copy(x))
        grad = grad_f(x, a, b)
        push!(grads, copy(grad))
    end
    return x, history, grads
end

function steepest_descent_general(x0::Vector, func::Function,
    grad_func::Function; max_iter::Int=1000, tol::Real=1e-8)
    history = [copy(x0)]
    grads = []
    x = copy(x0)
    grad = grad_func(x)
    push!(grads, copy(grad))
    for _ in 1:max_iter
        if norm(grad) < tol
            break
        end
        d = -grad
        alpha = backtracking_line_search(x, d, grad, func)
        if alpha <= 1e-12
            break
        end
        x = x + alpha * d
        push!(history, copy(x))
        grad = grad_func(x)
        push!(grads, copy(grad))
    end
    return x, history, grads
end

function check_orthogonality(grads::Vector)
    vals = Float64[]
    for i in 1:length(grads)-1
        g1 = grads[i]
        g2 = grads[i+1]
        n1 = norm(g1)
```

```
        n2 = norm(g2)
        if n1 > 1e-12 && n2 > 1e-12
            push!(vals, abs(dot(g1, g2)) / (n1 * n2))
        end
    end
    return vals
end

function plot_comparison(a::Real, b::Real, x0::Vector, histories::Vector, method_names::Vector)
    x_range = LinRange(-15, 15, 100)
    y_range = LinRange(-15, 15, 100)
    Z = [f([xi, yi], a, b) for yi in y_range, xi in x_range]
    colors = [:red, :blue, :green, :orange]
    p = surface(x_range, y_range, Z, color=:viridis, alpha=0.7, camera=(45, 30),
                xlabel=" x  ", ylabel=" x  ", zlabel="f( x  ,  x  )",
                legend=:topright, legendfontsize=8,
                plot_title="                                    : a=$a, b=$b", size=(600, 500))
    z_max = maximum(Z)
    for (idx, history) in enumerate(histories)
        x_traj = [pt[1] for pt in history]
        y_traj = [pt[2] for pt in history]
        z_traj = [f(pt, a, b) for pt in history]
        z_offset = (idx - 1) * 0.02 * max(z_max, 1.0)
        plot!(p, x_traj, y_traj, z_traj .+ z_offset, label=method_names[idx], color=colors[idx],
              linewidth=2.5, marker=:circle, markersize=3)
    end
    scatter!(p, [x0[1]], [x0[2]], [f(x0, a, b)], label="                ", color=:black, markersize=8, markershape=:star5)
    scatter!(p, [0], [0], [0], label="                ", color=:yellow, markersize=10, markerstyle=:diamond)
    return p
end

function plot_comparison_general(x0::Vector, func::Function, func_name::String,
                                 x_range::AbstractRange, y_range::AbstractRange,
                                 opt_point::Vector, histories::Vector, method_names::Vector{String})
    colors = [:red, :blue, :green, :orange]
```

```
    method_symbols = [:circle, :square, :diamond, :pentagon]
    Z = [func([x, y]) for y in y_range, x in x_range]
    p = surface(x_range, y_range, Z, title="3D: $func_name",
                xlabel=" x  ", ylabel=" x  ", zlabel="f(x)",
                color=:viridis, alpha=0.7, camera=(45, 30),
                legend=:topright, size=(600, 500))
    for i in 1:length(histories)
        hist = histories[i]
        x_vals = [point[1] for point in hist]
        y_vals = [point[2] for point in hist]
        z_vals = [func(point) for point in hist]
        plot!(p, x_vals, y_vals, z_vals, linewidth=2.5, color=colors[i],
              marker=method_symbols[i], markersize=3, label=method_names[i])
    end
    scatter!(p, [x0[1]], [x0[2]], [func(x0)], marker=:star8, markersize=12, color=:black, label="            ")
    scatter!(p, [opt_point[1]], [opt_point[2]], [func(opt_point)],
             marker=:diamond, markersize=10, color=:yellow, label="            ")
    return p
end

function main()
    methods = [:polak_ribiere, :hestenes_stiefel, :dixon, :dai_yuan]
    method_names = ["          -          ", "              -              ", "            ", "          "]
    params = [(1.0, 10.0), (5.0, 1.0), (3.0, 7.0)]
    x0_quad = [10.0, 10.0]

    for (a, b) in params
        println("f( x  , x  ) = $(a)*  x    + $(b)*  x    ,  x   = ($(x0_quad[1]), $(x0_quad[2]))")
        histories = []
        for (method_idx, method) in enumerate(methods)
            x_opt, history, directions = conjugate_gradient_quadratic(copy(x0_quad), a, b;
                                                                      method=method, max_iter=1000, tol=1e-8)
            push!(histories, history)
            conj_ok, conj_max = check_conjugacy(directions, a, b)
            @assert conj_ok "
                        (d_i, A d_j)      0"
```

```
            f_opt = f(x_opt, a, b)
            n_iter = length(history) - 1
            @assert n_iter <= 2 "                                ($a,$b), $(method_names[method_idx]): 2                    , $n_iter"
            @printf("  %-20s | x* = [%8.5f, %8.5f] | f(x*) = %.2e | : %3d\n",
                    method_names[method_idx], x_opt[1], x_opt[2], f_opt, n_iter)
        end
        filename_base = "comparison_quad_a$(Int(a))_b$(Int(b))"
        p = plot_comparison(a, b, x0_quad, histories, method_names)
        savefig(p, filename_base * ".png")
        println()
    end
    x0_rosenbrock = [-1.2, 1.0]
    optimal_rosenbrock = [1.0, 1.0]
    println("                                    , x   = ($(x0_rosenbrock[1]), $(x0_rosenbrock[2]))")
    histories_ros = []
    for (method_idx, method) in enumerate(methods)
        x_opt, history, directions = conjugate_gradient_general(copy(x0_rosenbrock), f_rosenbrock, grad_f_rosenbrock;
                                                                 method=method, max_iter=1000, tol=1e-8)
        push!(histories_ros, history)
        conj_ok, conj_max = check_conjugacy_general(directions, history, x -> ForwardDiff.hessian(f_rosenbrock, x); tol=1e10)
        f_opt = f_rosenbrock(x_opt)
        n_iter = length(history) - 1
        @printf("  %-20s | x* = [%8.5f, %8.5f] | f(x*) = %.2e | : %3d\n",
                method_names[method_idx], x_opt[1], x_opt[2], f_opt, n_iter)
    end
    x_ros = LinRange(-2, 2.5, 100)
    y_ros = LinRange(-1, 4, 100)
    p_ros = plot_comparison_general(x0_rosenbrock, f_rosenbrock, "                    ",
                                    x_ros, y_ros, optimal_rosenbrock, histories_ros, method_names)
    savefig(p_ros, "comparison_rosenbrock.png")
    println()
    x0_schwefel = [300.0, 300.0]
    optimal_schwefel = [420.9687, 420.9687]
```

```
    println("                                    , x    = ($(x0_schwefel[1]), $(x0_schwefel[2]))")
    histories_sch = []
    for (method_idx, method) in enumerate(methods)
        x_opt, history, directions = conjugate_gradient_general(copy(x0_schwefel), f_schwefel, grad_f_schwefel;
                                                                method=method, max_iter=500, tol=1e-8)
        push!(histories_sch, history)
        conj_ok, conj_max = check_conjugacy_general(directions, history, x -> ForwardDiff.hessian(f_schwefel, x); tol=1e10)
        f_opt = f_schwefel(x_opt)
        n_iter = length(history) - 1
        @printf("  %-20s | x* = [%8.5f, %8.5f] | f(x*) = %.2e | : %3d\n",
                method_names[method_idx], x_opt[1], x_opt[2], f_opt, n_iter)
    end
    x_sch = LinRange(250, 450, 100)
    y_sch = LinRange(250, 450, 100)
    p_sch = plot_comparison_general(x0_schwefel, f_schwefel, "                    ",
                                    x_sch, y_sch, optimal_schwefel, histories_sch, method_names)
    savefig(p_sch, "comparison_schwefel.png")
    println()
    x0_rastrigin = [3.0, 2.0]
    optimal_rastrigin = [0.0, 0.0]
    println("                                    , x    = ($(x0_rastrigin[1]), $(x0_rastrigin[2]))")
    histories_rast = []
    for (method_idx, method) in enumerate(methods)
        x_opt, history, directions = conjugate_gradient_general(copy(x0_rastrigin), f_rastrigin, grad_f_rastrigin;
                                                                method=method, max_iter=500, tol=1e-8)
        push!(histories_rast, history)
        conj_ok, conj_max = check_conjugacy_general(directions, history, x -> ForwardDiff.hessian(f_rastrigin, x); tol=1e10)
        f_opt = f_rastrigin(x_opt)
        n_iter = length(history) - 1
        @printf("  %-20s | x* = [%8.5f, %8.5f] | f(x*) = %.2e | : %3d\n",
                method_names[method_idx], x_opt[1], x_opt[2], f_opt, n_iter)
```

```
    end
    x_rast = LinRange(-5, 5, 100)
    y_rast = LinRange(-5, 5, 100)
    p_rast = plot_comparison_general(x0_rastrigin, f_rastrigin, "",
                                     x_rast, y_rast, optimal_rastrigin, histories_rast, method_names)
    savefig(p_rast, "comparison_rastrigin.png")
end

main()
```

Результат представлен на рисунках 1– 3.

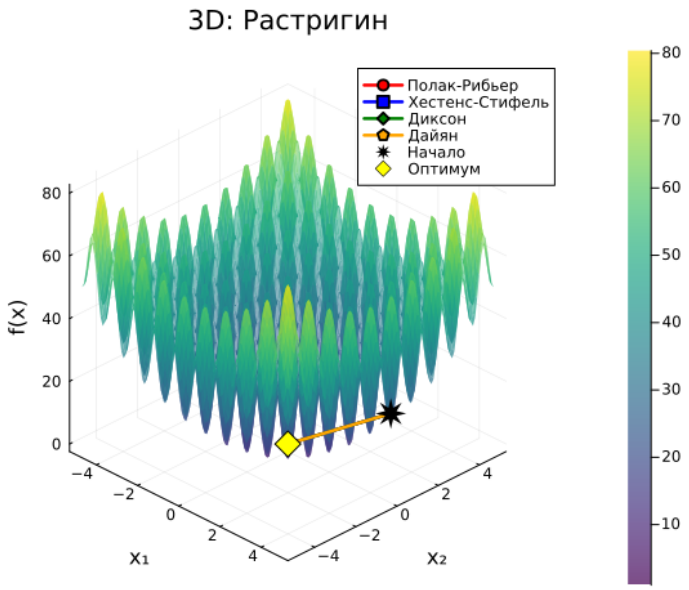

Рис. 1 — Результат

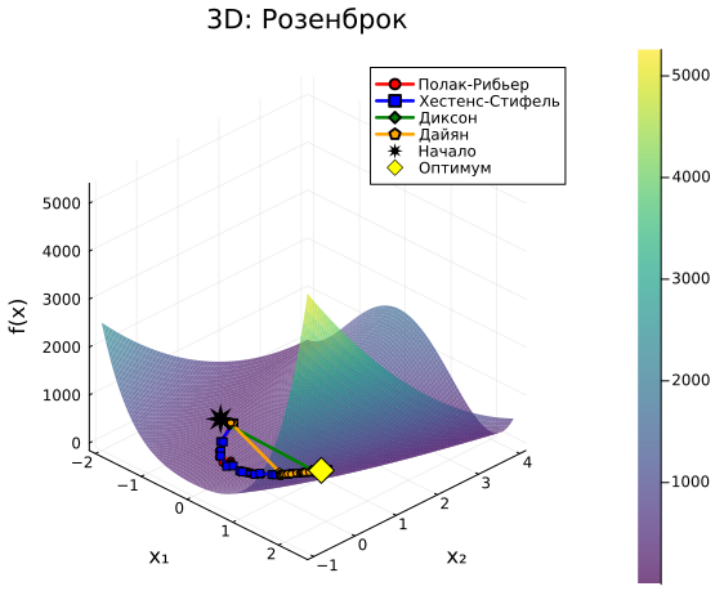

Рис. 2 — Результат

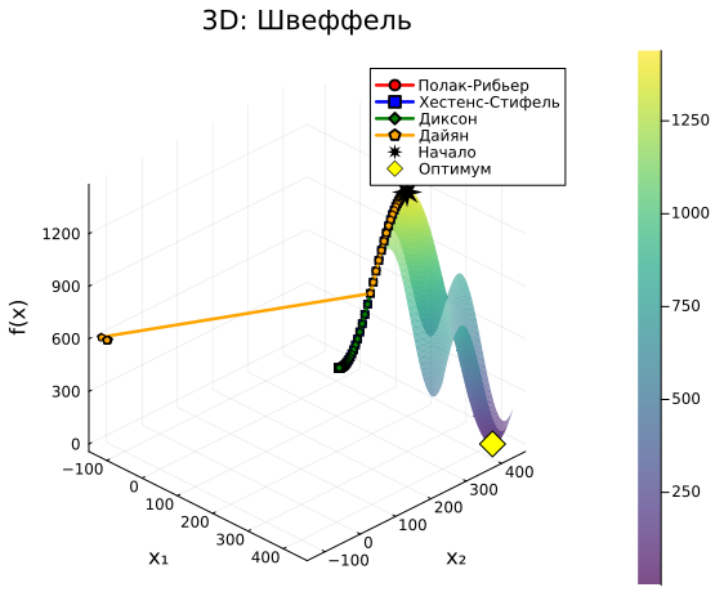

Рис. 3 — Результат